

## **ASSIGNMENTNO.12**

### **Title: Distributed Application using map reduce which processes a log file of a system**

---

#### **PROBLEM STATEMENT**

Design a distributed application using MapReduce to process a log file of a system.

#### **OBJECTIVES**

- To design a distributed application using the Apache Spark framework.
- To process a log file of a system using the MapReduce technique.

#### **OUTCOMES**

After completion of this assignment, students will be able to:

- Write code in Scala.
- Use RDD and SparkContext for parallel execution using the Spark framework.

#### **SOFTWARE REQUIREMENTS**

1. 64-bit open-source Linux (Ubuntu)
2. Java 8 or above
3. Apache Spark
4. Scala programming language

#### **THEORY**

##### **Application Programming Interface (API)**

Apache Spark provides its cluster computing capabilities through a library. This library is written in Scala, but its API is available in multiple languages such as Scala, Java, Python, and R. Developers can build Spark applications using any of these languages.

The Spark API consists of two important abstractions:

- SparkContext
- Resilient Distributed Datasets (RDDs)

These abstractions allow an application to connect to a Spark cluster and utilize cluster resources.

##### **SparkContext**

SparkContext is a class defined in the Spark library. It is the main entry point to the Spark framework. It represents a connection to a Spark cluster and is used to create other important objects in Spark.

A Spark application can have only one active instance of SparkContext at a time. To create another instance, the existing one must be stopped first.

### **Example:**

```
val sc = new SparkContext()
```

### **Resilient Distributed Datasets (RDDs)**

RDD is a distributed collection of data elements that can be processed in parallel. It is the core abstraction in Spark.

### ***Key Characteristics of RDD***

#### **1. Immutable**

Once created, an RDD cannot be modified. Any operation that changes data creates a new RDD.

#### **2. Partitioned**

The data in an RDD is divided into partitions and distributed across multiple nodes in a cluster.

#### **3. Fault Tolerant**

RDDs are fault tolerant. If a node fails, Spark can reconstruct lost data using lineage information.

#### **4. Interface**

RDD provides a common interface to process data from various sources such as HDFS, HBase, Cassandra, etc.

#### **5. Strongly Typed**

RDD stores homogeneous data types such as Integer, String, Float, etc.

#### **6. In-Memory Processing**

RDDs can be cached in memory, making data processing much faster.

### **Creating an RDD**

RDDs cannot be created directly because they are abstract. They are created using SparkContext or by transforming existing RDDs.

#### ***1. parallelize()***

Creates an RDD from a local collection:

```
val xs = (1 to 10000).toList  
val rdd = sc.parallelize(xs)
```

#### ***2. textFile()***

Creates an RDD from a text file:

```
val rdd = sc.textFile("hdfs://namenode:9000/path/to/file")
```

It also supports multiple files and compressed files:

```
val rdd = sc.textFile("hdfs://namenode:9000/path/*.gz")
```

### ***3. wholeTextFiles()***

Reads multiple text files as key-value pairs:

```
val rdd = sc.wholeTextFiles("path/to/data/*.txt")
```

- Key: File path
- Value: File content

### ***4. sequenceFile()***

Reads key-value pairs from sequence files stored in Hadoop-supported systems.

## **SCALA AND SPARK INSTALLATION STEPS**

1. Install Java:

```
java -version  
sudo apt install openjdk-8-jdk
```

2. Install Scala:

```
sudo apt install scala  
scala -version
```

### **3. Install Spark:**

```
wget https://downloads.apache.org/spark/spark-3.5.0/spark-3.5.0-bin-hadoop3.tgz  
tar -xvzf spark-3.5.0-bin-hadoop3.tgz  
sudo mv spark-3.5.0-bin-hadoop3 /opt/spark
```

4. Set Environment Variables in ~/.bashrc and run source ~/.bashrc

5. Verify using spark-shell

## **SAMPLE SCALA PROGRAM**

```
import org.apache.spark.SparkConf  
import org.apache.spark.SparkContext  
  
object LogFileAnalysis {  
  def main(args: Array[String]) {  
    val conf = new SparkConf().setAppName("Log Analysis").setMaster("local[*]")  
    val sc = new SparkContext(conf)  
  
    val logData = sc.textFile("log.txt")
```

```

val logLevels = logData.map(line => {
  if (line.contains("ERROR")) "ERROR"
  else if (line.contains("WARN")) "WARN"
  else if (line.contains("INFO")) "INFO"
  else "OTHER"
})

val counts = logLevels.map(level => (level, 1)).reduceByKey(_ + _)

counts.collect().foreach(println)
sc.stop()
}
}

```

## Steps to Run the Program

### 1. Compile the Program

```
scalac LogFileAnalysis.scala
```

### 2. Create JAR File

```
jar -cvf LogFileAnalysis.jar *.class
```

### 3. Run using Spark

```
spark-submit --class LogFileAnalysis LogFileAnalysis.jar
```

## Output Screenshots Section

### 1. Input File

- Screenshot of log.txt with sample data:

```

INFO Application started
ERROR Database failed
WARN Low memory
INFO User login

```

### 2. Compilation

- Screenshot of terminal showing:

```
scalac LogFileAnalysis.scala
```

### **3. Execution**

- Screenshot of:

```
spark-submit --class LogFileAnalysis LogFileAnalysis.jar
```

### **4. Output**

Expected output:

```
(INFO,2)
(ERROR,1)
(WARN,1)
```

### **Conclusion**

This experiment demonstrates how Apache Spark uses the MapReduce model to process log files efficiently using distributed computing and parallel execution with RDD.

Questions:

Q1. What is distributed system